SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:10 **Total Marks:50**

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

I **Jana Malakondaiah/192424223** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Jana Malakondaiah
Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes

of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

1. Smart City Surveillance System

Understanding of the Problem Context

The surveillance system continuously receives video streams from multiple cameras. Data moves between:

- I/O devices (cameras) → capture video.
- Main memory → stores frames temporarily.
- CPU → performs threat detection and analytics.

During peak hours, the amount of video data increases significantly, causing delays.

Application of Concepts

Interaction of CPU, Memory, and I/O

1. Cameras generate a large number of frames.
2. Frames are transferred through the system bus to memory.
3. CPU fetches the frames from memory and executes detection algorithms.
4. Processed results may again be written to memory or sent to output devices.

When many cameras transmit simultaneously:

- The bus becomes congested.
- The CPU waits for memory/I/O transfers.
- Memory bandwidth becomes insufficient.
- Overall threat detection response time increases.

Single-Bus Structure

- CPU, memory, and I/O share one common bus.
- Only one transfer can occur at a time.
- Simultaneous camera streams create contention.
- CPU often remains idle while waiting.

Result: Higher latency and lower throughput.

Multi-Bus Structure

- Separate buses are used for:

  1.CPU–memory communication

  2.I/O transfers

  3.Peripheral communication
- Multiple transfers can occur concurrently.

Advantages

- Reduced bus contention.
- Faster memory access.
- Better parallelism.
- Improved real-time performance.

Problem-Solving Approach

Improving Instruction Execution Cycle

The instruction cycle consists of:

1. Fetch
2. Decode
3. Execute
4. Memory access
5. Write-back

Performance can be improved by:

1. Pipelining

Multiple instructions are processed simultaneously in different stages.

2. Cache Memory

Frequently used video-processing data is stored closer to the CPU.

3. DMA (Direct Memory Access)

Camera data is transferred directly to memory without excessive CPU involvement.

4. Parallel Processing

Multiple cores process different video streams simultaneously.

Accuracy of Solution

Recommended Architecture

Feature	Effect
Multi-bus organization	Reduces transfer bottlenecks
DMA	Lowers CPU overhead
Cache memory	Reduces memory access time
Pipelined CPU	Increases instruction throughput
Multi-core	Handles multiple camera feeds concurrently

Conclusion

The lag occurs because heavy I/O traffic competes with CPU and memory accesses. A single-bus system becomes a bottleneck, whereas a multi-bus architecture, DMA support, cache memory, and pipelined execution significantly improve throughput and reduce threat-detection delay.

2. Real-Time Stock Trading Application

Understanding of the Problem Context

The trading application executes millions of instructions per second. During heavy trading:

- CPI (Cycles Per Instruction) increases.
- Memory accesses become frequent.
- Branch instructions cause stalls.
- Transaction processing is delayed.

Application of Concepts

Fetch–Decode–Execute Cycle

For every instruction:

1. Fetch instruction from memory.
2. Decode instruction.
3. Execute operation.
4. Access memory if required.
5. Write result back.

Execution time is:

Execution Time = Instruction Count × CPI × Clock Cycle Time

A high CPI directly increases execution time.

Causes of High CPI

- Cache misses.
- Frequent database/memory accesses.
- Conditional branches in trading logic.
- Pipeline stalls and flushes.

Problem-Solving Approach

Methods to Reduce CPI

1. Increase Cache Efficiency

- Larger L1/L2 cache.
- Better replacement policies.

2. Branch Prediction

Predict branch outcomes to avoid pipeline flushing.

3. Instruction Pipelining

Overlap fetch, decode, and execute stages.

4. Out-of-Order Execution

Execute independent instructions while waiting for memory.

5. Superscalar Architecture

Issue multiple instructions per clock cycle.

6. Prefetching

Load likely-needed data into cache before use.

Accuracy of Solution

Expected Improvements

Enhancement	**Benefit**
Better cache	Fewer memory stalls
Branch prediction	Reduced branch penalty
Pipelining	Higher throughput
Out-of-order execution	Better CPU utilization
Superscalar execution	More instructions completed per cycle

Conclusion

The delay is mainly caused by a high CPI resulting from memory stalls and branch penalties. By improving cache performance, branch prediction, pipelining, and instruction-level parallelism, the processor can execute transactions faster and provide better real-time responsiveness.

3. 8085 Temperature Control Unit

Understanding of the Problem Context

An 8085-based temperature controller reads sensor values and controls actuators. Incorrect readings occur occasionally due to improper addressing mode usage.

Application of Concepts:

Addressing Modes in 8085

Immediate Addressing

Data is included in the instruction.

Example:

MVI A,25H

Register Addressing

Data is stored in a register.

MOV A,B

Direct Addressing

The memory address is specified explicitly.

LDA 2050H

Register Indirect Addressing

Address is stored in a register pair.

MOV A,M

Problem-Solving Approach

Possible Causes of Incorrect Readings

1. Wrong memory address used.
2. HL pair not initialized properly.
3. Sensor data overwritten accidentally.
4. Using immediate data instead of actual sensor data.
5. Timing mismatch between sensor update and data read.

Reliable Programming Practices

- Initialize all register pairs.
- Use direct addressing for fixed sensor locations.
- Validate sensor data range.
- Store readings safely before processing.
- Use proper flags and conditional instructions.

Accuracy of Solution

Example

```
LDA 2050H      ; Read sensor value
CPI 64H        ; Compare with limit
JC NORMAL
MVI A,01H      ; Activate cooling
STA 3000H
```

This ensures:

- Correct memory access.
- Proper comparison.
- Reliable actuator control.

Conclusion

Addressing modes determine where data is obtained from. Incorrect selection can cause wrong sensor readings. Careful use of direct and register-indirect addressing, proper initialization, and validation routines ensures accurate and reliable temperature control.

Q4. 8086 Segmentation and Program Execution

Understanding of the Problem Context

The 8086 system runs multiple programs simultaneously using segmented memory. Problems include:

- Incorrect data access.
- Stack overflow.
- Program instability.

Application of Concepts:

8086 Segments

Segment	Register	Purpose
Code Segment	CS	Instructions
Data Segment	DS	Variables
Stack Segment	SS	Stack operations

Physical address:

Physical Address = Segment × 10H + Offset

Instruction Execution

- Instructions are fetched using CS.
- Data is accessed through DS.
- Stack uses SS.

Problem-Solving Approach:

Causes of Errors

Incorrect Data Access

- DS points to the wrong segment.
- Offsets exceed valid range.

Stack Overflow

- Excessive PUSH/CALL operations.
- SP not initialized correctly.
- Missing POP/RET instructions.

Correct Management

Initialize Segments

```
MOV AX,DATA
MOV DS,AX
```

Initialize Stack

```
MOV AX,STACK
MOV SS,AX
MOV SP,0FFFEH
```

Use Proper Addressing Modes

- Based
- Indexed
- Based-indexed

Accuracy of Solution

Example

MOV BX,ARRAY

MOV AL,[BX+SI]

This accesses the intended array element safely.

Conclusion

Segmentation separates code, data, and stack memory. Improper segment initialization or stack management causes incorrect accesses and overflow errors. Correct segment setup, stack initialization, and addressing-mode usage ensure reliable execution.

Q5. Hexadecimal–Binary Conversion in Data Communication

Understanding of the Problem Context

Packet data is represented in hexadecimal form. A software engineer misinterprets values, causing incorrect processing.

Application of Concepts:

Importance of Hexadecimal and Binary

Computers execute binary instructions, while hexadecimal provides a compact representation.

Example:

- Hex 3A
- Binary 00111010

Each hexadecimal digit represents exactly 4 bits.

Why Conversion Matters

Packet fields, addresses, flags, and opcodes are interpreted bit-by-bit. A wrong conversion changes the meaning of data.

Problem-Solving Approach

Example of Error

Suppose:

- Correct hex: AF
- Correct binary: 10101111

If interpreted incorrectly as:

- 11110000
- Packet flags change.
- Addresses become incorrect.
- CPU may execute wrong operations.

Effect on Instruction Execution

- Wrong opcode fetched.
- Incorrect branch decision.
- Data corruption.
- Communication failure.

Accuracy of Solution:

Detecting and Minimizing Errors

1.Efficient CPU Design

- Parity/ECC checking.
- Exception handling.
- Hardware validation.

2.Benchmarking

Benchmark tests measure:

- Error rate.
- Latency.
- Throughput.
- Reliability under load.

3.Real-Time Monitoring

- Packet verification.
- Checksum validation.
- Automated logging.

Conclusion

Accurate hexadecimal-to-binary conversion is essential because CPUs ultimately process binary data. Conversion errors can alter packet contents and instruction execution. Efficient CPU design, error-checking hardware, benchmarking, and real-time validation help detect and minimize such issues in communication systems.